

Optimizers in Deep Learning

1. Introduction

When we train a neural network, it tries to **learn patterns** from data by adjusting **weights and biases**.

But how does it decide *how much to change these weights*?

That's where **Optimizers** come in.

An **optimizer** is an algorithm that updates the model's parameters (weights) to **reduce the loss function** — i.e., make predictions more accurate.

So, in short:

Optimizers help neural networks *learn faster and better* by finding the minimum error.

2. What are Optimizers?

Every neural network has:

- **Weights (W)** → control the strength of connections
- **Biases (b)** → help shift the activation

During training, we calculate a **loss** — a number showing how wrong our predictions are.

The optimizer's job is to **tweak the weights** to make that loss smaller after every iteration.

It does this using **Gradients**, found by **Backpropagation**.

So the process looks like this:

1. Forward pass → compute output and loss
 2. Backward pass → compute gradient (how much each weight affects loss)
 3. Optimizer updates weights → reduce loss
-

3. Role of Optimizers

Optimizers control **how fast and in what direction** the model learns.

If we think of training as finding the **lowest point in a valley (minimum loss)**:

- The **loss function** = the valley
- The **gradient** = slope of the hill
- The **optimizer** = how we move down the hill

Without a good optimizer:

- The model might get stuck halfway (local minima)
 - It might take forever to reach the best solution
 - Or it might overshoot and never settle (unstable training)
-

4. The Challenges in Optimization

When training deep networks, we face a few common problems:

Problem	Explanation
Local Minima	The optimizer may get stuck in a small dip (not the global best).
Saddle Points	Flat areas where gradients are zero — training slows down.
Vanishing / Exploding Gradients	Gradients become too small or too large — common in deep networks.
Choosing the Right Learning Rate	Too high → unstable training; too low → slow learning.
Overfitting / Underfitting	Model learns too much noise or too little pattern.

Optimizers are designed to deal with these issues efficiently.

5. Types of Optimizers

Let's understand each major type step-by-step

1. Gradient Descent (GD) - The Basic Idea

It's the foundation of all optimizers.

Formula:

$$w = w - \alpha \frac{\partial L}{\partial w}$$

Where:

- w = weight
- α = learning rate
- $\frac{\partial L}{\partial w}$ = gradient of the loss

It moves weights *in the opposite direction* of the gradient to reduce loss.

Problem: It uses **all data at once** (batch gradient descent) — very slow for large datasets.

2. Stochastic Gradient Descent (SGD)

Instead of all data, SGD updates weights **after each sample**.

Pros:

- Faster updates
- Introduces randomness → helps escape local minima

Cons:

- Can be noisy → loss curve jumps a lot

It's like walking down a hill with some noise, you may wobble, but you'll reach faster.

3. Mini-Batch Gradient Descent

A practical version:

- Takes a **small batch (subset)** of data at a time
- Combines the speed of SGD with stability of Batch GD

Most modern optimizers use this approach.

4. Momentum Optimizer

- Sometimes, GD oscillates especially when the surface is steep.
- Momentum adds a “**memory**” to smooth the updates.

Idea: Combine current gradient with a fraction of previous update.

This helps build *speed in the right direction* and reduces zig-zag motion.

Analogy:

Imagine rolling a ball down a hill, momentum keeps it going smoothly.

5. RMSProp (Root Mean Square Propagation)

1. RMSProp adjusts learning rate **individually for each weight**.
 2. It keeps track of how fast each weight's gradient changes and scales it down accordingly.
 3. Works great for **non-stationary data** (like RNNs)
 4. Solves vanishing gradient to some extent
 5. Needs parameter tuning (decay rate, etc.)
-

6. Adam (Adaptive Moment Estimation)

This is the **most popular optimizer** in Deep Learning.

It combines:

- **Momentum** (from past gradients)
- **RMSProp** (adaptive learning rates)

So Adam adjusts:

- The *direction* using momentum
- The *speed* using adaptive learning rates

Fast convergence

Works well in most tasks

Requires less tuning

Sometimes overshoots or doesn't generalize as well as SGD

Default parameters (good start):

$$\text{learning rate} = 0.001, \beta_1 = 0.9, \beta_2 = 0.999$$

7. AdamW (Adam with Weight Decay)

1. An improved version of Adam that adds **regularization**.
 2. Prevents overfitting and ensures better generalization.
 3. Mostly used in modern deep networks like **Transformers**.
-

6. How to Choose the Right Optimizer?

Situation	Best Choice
Small/simple models	SGD
Large datasets or deep networks	Adam
Recurrent Neural Networks	RMSProp
When you face overfitting	AdamW / SGD with weight decay
Want high accuracy but can wait	SGD with Momentum

Tip: Start with **Adam** for experimentation.

Once the model works, try **SGD + Momentum** to fine-tune performance.

7. Summary / Key Takeaways

- Optimizers are algorithms that **update weights** to minimize loss.
 - They work hand-in-hand with **loss functions** and **backpropagation**.
 - **Gradient Descent** is the base idea → other optimizers build on it.
 - **SGD, Momentum, RMSProp, Adam** are most commonly used.
 - Choosing the right optimizer impacts **speed, accuracy, and stability** of training.
-

Quick Analogy:

Imagine training a neural network like hiking down a foggy mountain to reach the lowest point (minimum loss):

- **Gradient Descent:** one big, careful step each time
 - **SGD:** small, quick steps with some randomness
 - **Momentum:** use past direction to move smoother
 - **RMSProp:** adjust step size depending on terrain
 - **Adam:** combines memory + adaptability — smart hiking!
-